

EE-170L Computer Programming Lab

Laboratory Session 09

Introduction to Functions in C++

Objective

The students will learn and apply the

- Introduction to Function
- Prototype of Function
- Examples

1 Introduction to Functions

Functions let you chop up a long program into named sections so that the sections can be reused throughout the program. Function has a name identifier, accepts parameters and returns a result. C++ functions can accept an unlimited number of parameters. In general, C++ does not care in what order you put your functions in the program, so long as the function name is known to the compiler before it is called.

We have been consistently using library functions in the programs such as **getch()** and **clrscr()**. We just simply use these functions without knowing what's inside these functions. We just call them from our program with some parameters inside the brackets and it does the work for us. We don't know its implementation. When we actually call these functions, the code inside these functions executes and at the end it transfers the control back to the program from where the function was called.

In this lab we would learn how to make our own functions which can perform certain tasks. Function is differentiated from a simple variable, by the parenthesis just after the name as **main()**. It just indicates to the compiler that it's a function.

The most common function that you have encountered is **main()** which you have been implementing throughout the lab. **main()** is the function whose name identifier is main with no parameters passed to it. Anything inside brackets are the parameters passed to the function.

In case of multiple parameters, each of them are separated with a comma(,) in between. Parameters are written as the comma separated list within brackets just after the name of function. In **main()** function there are no parameters passed to it as the brackets after the name is empty.

As you know when we use **int main()** in our program then we return an integer from the main function. Similarly when we define our own functions in the program, that function should return appropriate value according to the data type which is given before the name of function when we give its definition or implementation, just as we return an integer when we use **int main()** and nothing in case of **void main()**. C++ assumes that every function will return a value. If the programmer wants a return value, this is achieved using the return statement. If no return value is required, none should be used when calling the function.

2 Prototype of Function

The prototype of a function is a way of describing the parameters (also called as arguments of function) and parameter types with which a legal call to the function can be made. It contains the name of the function, its parameters and their type, and the return value.

Syntax:

returnType functionName(List of parameters separated by commas);

Examples:

```
int main();  
float divide(int a, int b);  
void square(float x);
```

Here names of the functions are **main**, **divide** and **square** which take 0, 2 and 1 parameters respectively.

Example:

In this example main function just calls the function named asterisks which just prints the line of ten asterisks.

```
1  #include <iostream>  
2  using namespace std;  
3  
4  void asterisk(); // function prototype  
5  int main()  
6  {  
7      asterisk(); // function call  
8      return 0;  
9  }  
10 // function definition  
11 void asterisk()  
12 {  
13     for (int i = 0; i < 10; i++)  
14     {  
15         cout << "*";  
16     }  
17 }
```

LabCodes/lab10p01.cpp

The **asteriks()** function in the previous example does not take any argument and does not return any thing represented by empty braces **()** after function name and **void** before function name respectively.

Example:

Lets go one step ahead, function asterisks (int a) with a single argument.

```
1  #include <iostream>  
2  using namespace std;  
3  
4  void asterisk(int n); // function prototype  
5  int main()  
6  {  
7      asterisk(12); // function call  
8      return 0;  
9  }
```

```
10 // function definition
11 void asterisk(int num)
12 {
13     for (int i = 0; i < num; i++)
14     {
15         cout << "*";
16     }
17 }
```

LabCodes/lab10p02.cpp

This function would take an integer number as an argument and prints the number of asterisks equal to the number given as argument to the function in straight line.

As it is given in function prototype, compiler expects function definition should take single argument which should be of integer data type. Due to this in function calling you need to give an integer as argument (parameter) of a function.

Example:

Now an example of a function with two parameters.

```
1  #include <iostream>
2  using namespace std;
3
4  int add(int, int); // function prototype
5  int main()
6  {
7      int result;
8      // function call
9      result = add(3, 5);
10
11     cout << result;
12
13     return 0;
14 }
15 // function definition
16 int add(int num1, int num2)
17 {
18     return num1 + num2;
19 }
```

LabCodes/lab10p03.cpp

Lab09:Lab Tasks

1. Function Introduction and Declaration

- Declare a function called "greet" that takes no parameters and returns nothing.
- Inside the function, display a greeting message of your choice.
- Call the function from the main program to execute the greeting.

2. Basic Function Declaration and Call

- Declare a function called "sayHello" that takes no parameters and returns nothing.
- Inside the function, display the message "Hello, World!".
- Call the function from the main program to execute the greeting.

3. Write a C++ program that:

Takes **10 integer values** as input from the user and stores them in a **1D array**.

Creates a function named calculateSum that:

Accepts the array and its size as parameters

Returns the **sum** of all elements

In the main() function:

Call the function

Calculate and display the **average** of the array elements

Constraints:

Use only one-dimensional array

Use function for sum calculation

